

Django REST Framework (REST API Development)

- Module 5

Interview Questions with Detailed Answers

1. What is an API and why are APIs important in web development?

API (Application Programming Interface) allows communication between different software systems. APIs are important because they enable applications to exchange data, integrate services, and support mobile, web, and third-party applications.

2. Explain what a REST API is and name two key principles it follows.

REST API is an API based on Representational State Transfer architecture. Key principles: (1) Stateless communication and (2) Resource-based URLs using standard HTTP methods.

3. Difference between JSON and XML?

JSON is lightweight, human-readable, and widely used in modern APIs. XML is more verbose, supports attributes and namespaces, and is heavier than JSON.

4. How is DRF different from standard Django?

DRF extends Django with serializers, API views, authentication, permissions, pagination, routers, and tools for building RESTful APIs.

5. Steps to install DRF?

`pip install djangorestframework`, add 'rest_framework' to `INSTALLED_APPS`, run migrations, and start creating serializers, views, and URLs.

6. Purpose of adding 'rest_framework' to INSTALLED_APPS?

It enables DRF functionality including serializers, authentication, API views, browsable API, and configuration settings.

7. Difference between Serializer and ModelSerializer?

Serializer requires manual field definitions. ModelSerializer automatically generates fields and validation from a Django model.

8. Create Hello API using APIView.

```
from rest_framework.views import APIView
from rest_framework.response import Response
```

```
class HelloAPIView(APIView):
    def get(self, request):
        return Response({'message': 'Hello API'})
```

9. What is a Django model?

A model represents a database table. Models define data structure and are created before APIs because APIs usually expose model data.

10. Serializer for Doctor model.

```
from rest_framework import serializers
```

```
class DoctorSerializer(serializers.Serializer):  
    name = serializers.CharField(max_length=100)  
    specialization = serializers.CharField(max_length=100)
```

11. CRUD API using APIView.

Implement GET, POST, PUT, DELETE methods inside APIView to perform Create, Read, Update, and Delete operations.

12. Difference between 200 and 201?

200 OK indicates successful request. 201 Created indicates a new resource was successfully created.

13. Return JSON response from DRF endpoint.

```
from rest_framework.response import Response
```

```
return Response({'status':'success'})
```

14. POST endpoint using ModelSerializer.

```
class DoctorCreateAPIView(APIView):  
    def post(self, request):  
        serializer = DoctorSerializer(data=request.data)  
        if serializer.is_valid():  
            serializer.save()  
        return Response(serializer.data, status=201)  
        return Response(serializer.errors, status=400)
```

15. Use of GenericAPIView and Mixins.

They reduce code duplication by providing reusable CRUD behavior.

16. Update Doctor using PUT.

Fetch object, pass instance and request data to serializer, validate, save, and return updated data.

17. What is ViewSet?

ViewSet groups related CRUD operations into one class. APIView requires separate methods and more code.

18. Purpose of Router?

Router automatically generates URL patterns for ViewSets.

19. DefaultRouter vs SimpleRouter.

DefaultRouter adds API root view; SimpleRouter only generates routes.

20. How pagination helps?

It improves performance, reduces response size, and makes data easier to consume.

21. Implement PageNumberPagination.

```
from rest_framework.pagination import PageNumberPagination
```

```
class DoctorPagination(PageNumberPagination):  
    page_size = 10
```

22. LimitOffsetPagination vs CursorPagination.

LimitOffset uses limit and offset values. CursorPagination uses encoded cursor values and is better for large datasets.

23. Add ordering by name.

```
from rest_framework.filters import OrderingFilter  
filter_backends = [OrderingFilter]  
ordering_fields = ['name']
```

24. Main authentication types in DRF.

SessionAuthentication, TokenAuthentication, JWT Authentication, and BasicAuthentication.

25. TokenAuthentication vs SessionAuthentication.

TokenAuthentication uses token in headers. SessionAuthentication uses server-side sessions and cookies.

26. Permission class for authenticated users.

```
from rest_framework.permissions import IsAuthenticated  
permission_classes = [IsAuthenticated]
```

27. Protect /api/doctors/ endpoint.

Apply IsAuthenticated permission class to the view or globally in DRF settings.

28. Role of IsAdminUser.

It grants access only to admin/staff users.

29. Custom permission for specialization.

Create custom permission class and override has_object_permission() to validate specialization.

30. Generate and use authentication token.

Install DRF token auth, create token using `Token.objects.create(user=user)`, send token in Authorization header.

31. What is JWT?

JWT is a signed token containing encoded user data. Unlike basic token authentication, JWT can store claims and supports expiration.

32. Using ChatGPT/Copilot for explanations.

Use AI to generate explanations but verify results with official DRF documentation.

33. Integrate external API.

Install requests package, call external API, parse JSON response, and return data through DRF endpoint.

34. Weather API endpoint.

```
import requests
from rest_framework.views import APIView
from rest_framework.response import Response

class WeatherAPIView(APIView):
    def get(self, request, city):
        data = requests.get('API_URL').json()
        return Response(data)
```

35. Handle external API errors.

Use try-except blocks, timeouts, validation, and return proper HTTP error responses.

36. What is Postman?

Postman is an API testing tool used to send requests, inspect responses, automate tests, and manage collections.

37. Use Postman AI or ChatGPT.

Generate code snippets and verify endpoints, authentication, parameters, and response formats.

38. Send email using Mailgun/MailChimp.

Configure API credentials, send HTTP request from DRF endpoint, and handle response status.

39. Integrate Twilio SMS.

Install Twilio SDK, configure credentials, and call Twilio client to send SMS messages.

40. Deploy DRF API to PythonAnywhere.

Upload project, create virtual environment, install dependencies, configure WSGI, database, static files, and restart app.

41. Why API versioning?

It prevents breaking existing clients. Example: /api/v1/doctors/ and /api/v2/doctors/.

42. Custom JSON response format.

```
return Response({  
    'result': serializer.data  
})
```

43. Importance of exception handling.

It improves API reliability, provides meaningful error messages, and prevents crashes.

44. Troubleshoot 500 error.

Check logs, traceback, database connectivity, recent code changes, serializers, and API requests.

45. Secure payment API endpoint.

Use authentication, permissions, HTTPS, validation, exception handling, and secure storage of API keys.

46. Paginated authenticated doctor endpoint.

Use IsAuthenticated, PageNumberPagination, filtering backend, and specialization filters.

47. Upgrade API to v2 safely.

Use API versioning, maintain v1 support, add new serializer/model changes in v2, and provide migration documentation.